Pocket expense - file upload

1. End-to-end flow

1. User navigates to Out-of-Pocket Expenses Batch Upload.
2. User selects “User whose out-of-pocket expenses you are uploading” from a dropdown.
   - Dropdown shows only users this admin can manage (or all for primary admin).
   - For “Business/Card User” context, default the dropdown to the logged-in user.
3. User downloads CSV template, fills it, then uploads completed file.
4. System does basic file checks immediately (size, extension etc).
5. If basic checks pass:
   - File and “selected user” info are saved as upload with default status
   - CSV is parsed and Row level validation validates all entries
     - its a synchronous process so all errors to be captures as array/json and sent back as response
     - if all entries are good, the expense records should be created with number of entries and success message
     - a notification should be issues once all expenses are created for the user
6. For Error screen User can review result:
   - each line of error should be displayed for the user with line number, error field and error reason
   - this list should be downloadable as csv for user to keep as a future reference

	CSV column	Target DB field	Notes / assumptions	validation rule / Questions
1	Date	`pocket_expense.date`	Required dd-mm-yyyy	Required, not less than 3 years old
2	Expense Type	`opt_pocket_expense_t`	Required	Must match dropdown

		`ype.option`	foreign key validation **+ve or -ve amount validation based on type selection**	values one of 1. ATM Withdrawal 2. Point of Sale 3. Fee & Charges 4. Refund from Merchant
3	Currency Code	`pocket_expense.currency`	Required 3-letter ISO.	Must match dropdown values
4	Amount	`pocket_expense.amount`	Required; decimal(14,2).	**add sign +ve or -ve based on type selection**
5	{Currency} Equivalent Amount	`pocket_expense.user_converted_amount`	optional	**add sign +ve or -ve based on type selection**
6	VAT %	`pocket_expense.vat_amount`	optional	optional **vat_amount** < 100
7	Merchant Name	`pocket_expense.merchant_name`	Required	
8	Description	`pocket_expense.merchant_description`	optional	

9	Merchant Address	`pocket_expense.merchant_address`	Optional.	
10	Merchant Country	`pocket_expense.merchant_country`	optional	Must match dropdown values

11	Source	from table
		`pocket_expense_source_client_config.name`
		for the client_id
		store at
		`pocket_expens_metadata`
			optional
			when “Other”. Source Note must be filled
				Must match dropdown values
				when “Other”. Source Note must be filled

12	Source Note	store at `pocket_expens_metadata`	optional But Required when Source = other	optional but Required when
13	Notes	`pocket_expense.notes`	Optional free text.	follow DB limit, trim and prevent sql injection
14	none in CSV	`pocket_expense.user_`	Comes from **selected user** in	

		`id`	UI,	
15	(none in CSV)	`status`	`submitted`	
16	(none in CSV)	`created_by_user_id`	Logged-in user	
17	(none in CSV)	`updated_by_user_id`, `approved_by_user_id`, timestamps, deleted flags	Set internally, no CSV mapping.	

General Questions:

1. ~~How to display wallet base currency for file upload?~~
   a. new optional field to capture data, this can be edited in the edit form
2. ~~How to capture all metadata for file uploaded expenses ?~~

   This will be added from the edit form
3. ~~How could receipt be attached for each expense?~~ This will be added from the edit form
4. ~~Where is the menu / page placed for file upload? Under main navigation~~
5. ~~When a file upload is completed by a regular user, will there be any notification for approval? Who will get notification, manager user or primary admin or both ?~~ Notification for user when file upload has created all the the expenses
6. ~~What is the expected maximum number of expense?~~ **try out if 200 possible**

1. Overview

This change introduces **CSV-based bulk upload of out-of-pocket expenses** for a selected user.

Key capabilities:

- Admin selects a **target user** whose expenses will be created.
- Admin uploads a **CSV file** following a defined template (max 200 rows).
- API performs **synchronous validation and creation**:
  - Validates file headers and each row against business rules and master data (currencies, expense types, sources, countries).
  - If any row fails validation, **no expenses are created** and an error payload is returned.
  - If all rows pass validation, the system creates new `pocket_expense` (+ related metadata) records and returns a success payload.
- An audit record of each upload is stored in the new table `pocket_expense_file_uploads`.
- The frontend can show a detailed error list and allow users to **download errors as CSV**.

---

2. CSV File Definition

File constraints

- Format: `text/csv` (or `.txt` with CSV content).
- Max size: configurable (initially sized for ~200 rows).
- Max rows: 200 (header row excluded).
- Header row is mandatory and must exactly match the column names below.

Columns

> See above

Data Model

New Table: `pocket_expense_file_uploads`

```
CREATE TABLE `pocket_expense_file_uploads` (
  `id` bigint(20) unsigned NOT NULL AUTO_INCREMENT,
  `uuid` char(36) DEFAULT NULL,
  `user_id` bigint(20) unsigned NOT NULL COMMENT 'Target user for expenses',
  `client_id` bigint(20) unsigned NOT NULL COMMENT 'Client/company context',
  `created_by_user_id` bigint(20) unsigned NOT NULL COMMENT 'Admin who uploaded',
  `file_name` varchar(255) NOT NULL,
  `file_path` varchar(255) NOT NULL,
  `total_records` int(10) unsigned NOT NULL DEFAULT 0,
  `valid_records` int(10) unsigned NOT NULL DEFAULT 0,
  `validation_errors` json DEFAULT NULL COMMENT 'Structured error data',
  `status` enum(
      'uploaded',
      'validation_failed',
      'validation_passed',
```

```
      'processing',
      'completed',
      'failed',
      'sync_failed'
  ) NOT NULL DEFAULT 'uploaded',
  `uploaded_at` timestamp NULL DEFAULT NULL,
  `validated_at` timestamp NULL DEFAULT NULL,
  `processed_at` timestamp NULL DEFAULT NULL,
  `created_at` timestamp NULL DEFAULT NULL,
  `updated_at` timestamp NULL DEFAULT NULL,
  `deleted_at` timestamp NULL DEFAULT NULL,
  PRIMARY KEY (`id`),
  KEY `pocket_expense_file_uploads_user_id_index` (`user_id`),
  KEY `pocket_expense_file_uploads_client_id_index` (`client_id`),
  KEY `pocket_expense_file_uploads_created_by_user_id_index` (`created_by_user_id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

```

- **Model**: `PocketExpenseFileUpload` (standard Eloquent model using soft deletes).
  - Status values typically used: `uploaded`, `validation_failed`, `validation_passed`, `processing` `completed`.

FPS API Contract

Route

```
// routes/api.php
Route::middleware([Oauth2UserClient::class])->group(function () {
    Route::post('/uploads/pocket-expense/csv', [PocketExpenseUploadController::class, 'uploadPocketExpenseCSV'])
        ->name('uploads.pocket-expense.csv');
});
POST /api/uploads/pocket-expense/csv
Content-Type: multipart/form-data
```

Form mandatory fields from frontend

- `file` (required): CSV file containing expenses.
- `user_id` (required): Current authenticated admin user (from token / context).
- `expense_user_id` (required): Target user for whom expenses are created.
- `client_id` (required): Client context id.

Server-side request validation

- `file`: `required|file|mimes:csv,txt|max:10240` (size tuned for 200 rows).
- `user_id`: `required|exists:users,id`.
- `expense_user_id`: `required|exists:users,id`.

- `client_id`: `required|exists:clients,id`.

Additional checks (after basic validation):

- Header row presence and exact column names.
- `expense_user_id` belongs to `client_id`.
- Client has **OOP feature enabled**.
- `user_id` (current admin) has permission to manage `expense_user_id`.

File processing

1. **Initial validation & upload record**
   - After successful request validation:
     - CSV is stored on disk (e.g. `storage/app/pocket-expense-uploads`).
     - A `PocketExpenseFileUpload` row is created with:
       - `status = 'uploaded'`
       - `user_id = expense_user_id`
       - `client_id`, `created_by_user_id`, `file_name`, `file_path`, `uploaded_at`.
2. **CSV parsing & row-level validation**
   - `PocketExpenseCSVValidator` service is instantiated with:
     - `targetUserId`, `clientId`, `adminId`.
   - Service:
     - Parses CSV into rows (excluding header) **max 200 rows**.
     - Iterates each row, running datatype and cross-field validations.
     - Accumulates: `errors`: array of structured error objects.
     - **All-or-nothing**: Either all valid rows succeed for storage of pocket expense or nothing
3. **Validation result persistence**
   - `PocketExpenseFileUpload` record is updated with:
     - `total_records` updated if file was read
     - `valid_records` updated if all records are valid and upload successful
     - `validation_errors` (JSON array of error objects) updated when there are any errors

    - `validated_at`
    - `status = 'validation_failed'` if any errors, otherwise remains ready to proceed.
4. **Decision**
    - If **any** row has errors:
        - No expenses are created.
        - API responds with **422** and error payload (see below).
    - If **all** rows are valid:
        - create `pocket_expense` record for each row with associated metadata (e.g. expense source, possibly VAT / additional info).
        - Update upload record: `status = 'completed' processed_at` set.
5. **Notification**
    - After successful creation of all expenses, issue notification to the target user/admin (channel TBD: in-app and/or email).

---

6. Validation Service Behaviour

6.1 dropdown values (implement caching if possible)

`PocketExpenseCSVValidator` preloads reference data for fast validation:

- Expense / transaction types (for `Transaction Type`).
- Supported currencies (for `Currency Code`).
- Supported countries (for `Merchant Country`).
- Expense sources configured for the client (for `Source`).

6.2 Key rules per row (summary)

- **Date**
    - Required.
    - Format: `DD/MM/YYYY`.
    - Optionally: not older than N years (e.g. 3 years).
- **Transaction Type**
    - Required.
    - Must exist in configured types.

  - Also used to determine **sign** of `Amount` (e.g. `Refund from Merchant` vs regular expense).
- **Currency Code**
  - Required.
  - Must be 3-letter ISO and in allowed list.
- **Amount**
  - Required.
  - Numeric.
  - Converted to signed value based on Transaction Type (refund vs expense).
- **VAT %**
  - Optional.
  - If present, must be numeric between 0–100 (after stripping `%`).
  - Contributes to VAT-related metadata.
- **Merchant Name**
  - Currently treated as required in validator, with max length check (per DB definition).
- **Source / Source Note**
  - If Source is provided:
    - Must match a configured source for client (including `Other`).
    - If Source = `Other`, then `Source Note` is required and length-validated.
- **Merchant Country**
  - Optional.
  - If present, must match allowed country list (same logic as Volopa API).
- **{Currency} Equivalent Amount**
  - Optional.
  - If present, must be numeric.

If any check fails for a row, that row is marked invalid and contributes to the `errors` list.

---

7. Error Response & Frontend Behaviour

7.1 Error response structure

When **any** validation error is found, the endpoint returns HTTP **422**:

```
{
  "success": false,
```

```
  "message": "Validation failed",
  "upload_id": 123,
  "total_rows": 100,
  "error_count": 3,
  "errors": [
    {
      "line_number": 2,
      "field": "Date",
      "error": "Invalid format. Use DD-MM-YYYY",
      "value": "2025-13-45"
    },
    {
      "line_number": 5,
      "field": "Transaction Type",
      "error": "Invalid expense type",
      "value": "Invalid Type"
    },
    {
      "line_number": 7,
      "field": "Source Note",
      "error": "Required when Source is \"Other\"",
      "value": ""
    }
  ]
}

```

- `line_number` corresponds to CSV line (including header), i.e. **header = line 1**.
- `errors` is also stored in `pocket_expense_file_uploads.validation_errors` for later retrieval/download.

Frontend requirements:

- Display each error row with:
  - Line number
  - Field name
  - Error reason
- Provide a **"Download errors as CSV"** option by re-formatting the `errors` array into a CSV for user reference.

7.2 Success response

When all rows are valid and expenses are created:

```
{
  "success": true,
  "message": "File validated successfully. Expenses created.",
  "upload_id": 123,
  "total_rows": 42
}

```

Storing Pocket expenses from File upload

```
1. Validate CSV → If errors → Return errors (stop)
2. If no errors → Store in local table (committed)
3. Dispatch background job immediately
4. Return success response
5. Background job syncs to main service in batches (batch of 100)
```

Local Storage schema

```database_schema
-- In upload service database
CREATE TABLE pocket_expense_uploads_data (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    upload_id BIGINT NOT NULL, -- FK to pocket_expense_file_uploads
    line_number INT NOT NULL,
    status ENUM('pending', 'processing', 'synced', 'failed') DEFAULT 'pending',

    -- All expense data (serialized or individual columns)
    expense_data JSON NOT NULL, -- Stores the complete validated expense

    created_at TIMESTAMP,
    updated_at TIMESTAMP,

    INDEX idx_upload_status (upload_id, status),
    INDEX idx_pending_sync (status) WHERE status IN ('pending', 'failed')
);

```

bulk insert to local table with single query for performance

```
$dataToInsert = [];
foreach ($validationResult['valid_rows'] as $row) {
    $dataToInsert[] = [
        'upload_id' => $uploadRecord->id,
        'line_number' => $row['line_number'],
        'expense_data' => json_encode($row['data']),
        'created_at' => now(),
        'updated_at' => now()
    ];
}

// Single insert query, better for performance
DB::table('pocket_expense_upload_data')->insert($dataToInsert);

//update file up table status to processing at this time
 $uploadRecord->update([
                'status' => 'processing'
  ]);

  ProcessExpenseUpload::dispatch($uploadRecord->id);

  // then send success response
```

---

8. Security & Permissions

- Endpoint is protected by `Oauth2UserClient` middleware.
- Additional server-side checks:
  - `user_id` must match authenticated user.
  - `expense_user_id` must belong to `client_id`.
  - Current user must have permission to manage `expense_user_id` (same rules as existing expense tooling).
  - Client must have **Out-of-Pocket** feature enabled.